

Lab 1: WEP Encryption

Network Security Laboratory Report

February 2026

Contents

1	Introduction	2
2	How WEP Works — Simple Overview	2
3	Step-by-Step Encryption	2
3.1	Step 1 Add a Checksum (ICV)	2
3.2	Step 2 — Generate the Keystream (RC4)	3
3.3	Step 3 — Scramble with XOR	3
4	Verification Decryption	3
5	Results Summary	4
6	Python Code & Explanation	4
6.1	Part 1 — Setting Up (Lines 1–6)	4
6.2	Part 2 — Computing the Checksum and Preparing the Data (Lines 8–12) .	4
6.3	Part 3 — The RC4 Functions (Lines 14–29)	5
6.4	Part 4 — Encryption, Sending, and Checking (Lines 31–45)	6
7	WEP Weaknesses	7
8	Conclusion	7

1 Introduction

WEP (Wired Equivalent Privacy) is an old security protocol used in Wi-Fi networks to keep data private. It works by scrambling the data before sending it so that only the intended receiver, who knows the secret key, can unscramble it.

In this lab, we encrypt the message "10101010" using WEP step by step, then write a Python program to do the same thing automatically and check that the results match.

The values we use are:

Parameter	Value
Plaintext (message)	"10101010"
IV (random number)	0xABCDEF
Secret Key	0x1327456789

2 How WEP Works — Simple Overview

WEP follows three main steps:

1. **Add a checksum to the message.** This lets the receiver check later that nothing was changed.
2. **Generate a secret scrambling sequence (keystream)** using the RC4 algorithm and the secret key.
3. **Scramble the message** by combining it with the keystream using XOR. The result is the encrypted message (ciphertext).

The receiver simply does the same process in reverse to get the original message back.

3 Step-by-Step Encryption

3.1 Step 1 Add a Checksum (ICV)

Before encrypting, we compute a checksum called CRC-32 over the message. Think of it as a fingerprint of the data. If anyone changes the data later, the fingerprint will no longer match.

$$\text{CRC-32}(\text{"10101010"}) = 0x6659081C \longrightarrow 1C\ 08\ 59\ 66\ (4\ \text{bytes})$$

We then attach these 4 bytes to the end of the message:

$$\underbrace{31\ 30\ 31\ 30\ 31\ 30\ 31\ 30}_{\text{message (8 bytes)}} \parallel \underbrace{1C\ 08\ 59\ 66}_{\text{checksum (4 bytes)}} = 12\ \text{bytes total}$$

We also prepare the key for the RC4 algorithm by joining the IV and the secret key together:

$$\text{RC4 Key} = AB\ CD\ EF\ 13\ 27\ 45\ 67\ 89$$

3.2 Step 2 — Generate the Keystream (RC4)

RC4 is the algorithm that produces the scrambling sequence. It has two parts:

Part A — KSA (setup): Creates a shuffled list of 256 numbers using the RC4 key. This shuffled list is the starting point for generating random-looking bytes.

Part B — PRGA (generation): Reads through the shuffled list and outputs one random-looking byte at a time. We need 12 bytes (one for each byte of our message + checksum).

Keystream = 02 B1 78 46 8F CF D1 65 9E D9 9A 6E

3.3 Step 3 — Scramble with XOR

XOR is a simple operation that combines two bits: same bits give 0, different bits give 1. We apply it byte by byte between our message+checksum and the keystream:

Byte	Message+Checksum	Keystream	Encrypted
1	0x31	0x02	0x33
2	0x30	0xB1	0x81
3	0x31	0x78	0x49
4	0x30	0x46	0x76
5	0x31	0x8F	0xBE
6	0x30	0xCF	0xFF
7	0x31	0xD1	0xE0
8	0x30	0x65	0x55
9	0x1C	0x9E	0x82
10	0x08	0xD9	0xD1
11	0x59	0x9A	0xC3
12	0x66	0x6E	0x08

Finally, we attach the IV at the front to form the WEP packet that gets sent over the network:

Packet = $\underbrace{\text{AB CD EF}}_{\text{IV (sent openly)}} \parallel \underbrace{\text{33 81 49 76 BE FF E0 55 82 D1 C3 08}}_{\text{encrypted data}}$

4 Verification Decryption

To check our work, we decrypt the packet. The receiver reads the IV from the packet, regenerates the same keystream using that IV and the shared secret key, then XORs again to get back the original message. Finally it recomputes the checksum and compares:

Recovered message	"10101010"
Checksum received	1C 08 59 66
Checksum recomputed	1C 08 59 66
Result	PASS — data is intact

5 Results Summary

Item	Value
Message	10101010
IV	AB CD EF
Secret Key	13 27 45 67 89
RC4 Seed	AB CD EF 13 27 45 67 89
Checksum (CRC-32)	1C 08 59 66
Message + Checksum	31 30 31 30 31 30 31 30 1C 08 59 66
Keystream	02 B1 78 46 8F CF D1 65 9E D9 9A 6E
Encrypted data	33 81 49 76 BE FF E0 55 82 D1 C3 08
WEP Packet	AB CD EF 33 81 49 76 BE FF E0 55 82 D1 C3 08

6 Python Code & Explanation

The Python program is split into four parts. Each part does one job.

6.1 Part 1 — Setting Up (Lines 1–6)

```
1 import struct
2 import binascii
3
4 IV          = bytes([0xAB, 0xCD, 0xEF])
5 SECRET_KEY  = bytes([0x13, 0x27, 0x45, 0x67, 0x89])
6 plaintext   = b'10101010'
```

This part loads two built-in Python tools and defines the three inputs:

- `struct` — used to convert a number into raw bytes.
- `binascii` — used to compute the CRC-32 checksum.
- `IV` — a 3-byte random-looking number attached to every packet so the keystream is different each time.
- `SECRET_KEY` — the 5-byte password shared between sender and receiver. It never leaves either device.
- `plaintext` — the message we want to encrypt.

6.2 Part 2 — Computing the Checksum and Preparing the Data (Lines 8–12)

```
1 icv_int      = binascii.crc32(plaintext) & 0xFFFFFFFF
2 icv_bytes    = struct.pack('<I', icv_int)
3 plaintext_icv = plaintext + icv_bytes
4 rc4_seed     = IV + SECRET_KEY
```

- `binascii.crc32(plaintext)` calculates the checksum (fingerprint) of the message. The `& 0xFFFFFFFF` part makes sure the result is always a positive number.
- `struct.pack('<I', icv_int)` turns that number into 4 bytes. The `<` means store the smallest byte first (little-endian), which is what WEP requires.
- `plaintext_icv` sticks the checksum onto the end of the message, giving us 12 bytes to encrypt.
- `rc4_seed` joins the IV and the secret key together into one sequence. This is what we feed into the RC4 algorithm.

6.3 Part 3 — The RC4 Functions (Lines 14–29)

```

1 def KSA(key):
2     S, j = list(range(256)), 0
3     for i in range(256):
4         j = (j + S[i] + key[i % len(key)]) % 256
5         S[i], S[j] = S[j], S[i]
6     return S
7
8 def PRGA(S, length):
9     S, i, j, ks = S[:], 0, 0, []
10    for _ in range(length):
11        i = (i + 1) % 256
12        j = (j + S[i]) % 256
13        S[i], S[j] = S[j], S[i]
14        ks.append(S[(S[i] + S[j]) % 256])
15    return bytes(ks)

```

KSA — the setup function:

- Starts with a list `S` containing the numbers 0 to 255 in order.
- Goes through the list 256 times and swaps pairs of numbers using the key to decide which pairs to swap.
- The result is the same list but in a scrambled order that depends entirely on the key. Two different keys will always produce two different scrambled lists.

PRGA — the keystream generator:

- Takes the scrambled list from KSA and keeps shuffling it to produce one output byte at a time.
- `S = S[:]` makes a copy of the list so the original is not changed.
- Each byte it outputs looks random but is completely determined by the key. Without knowing the key it is practically impossible to predict the next byte.
- It keeps going until it has produced exactly as many bytes as we need — 12 in this case.

6.4 Part 4 — Encryption, Sending, and Checking (Lines 31–45)

```
1 S = KSA(rc4_seed)
2 keystream = PRGA(S, len(plaintext_icv))
3 ciphertext = bytes(a ^ b for a, b in zip(plaintext_icv, keystream))
4 wep_frame = IV + ciphertext
5
6 recv_IV = wep_frame[:3]
7 recv_ciphertext = wep_frame[3:]
8 recv_seed = recv_IV + SECRET_KEY
9 S2 = KSA(recv_seed)
10 recv_keystream = PRGA(S2, len(recv_ciphertext))
11 decrypted = bytes(a ^ b for a, b in zip(recv_ciphertext,
    recv_keystream))
12 recv_plaintext = decrypted[:-4]
13 recv_icv = decrypted[-4:]
14 computed_icv = struct.pack('<I', binascii.crc32(recv_plaintext)
    & 0xFFFFFFFF)
```

Encryption side:

- `KSA(rc4_seed)` shuffles the list using our key.
- `PRGA(S, 12)` produces 12 scrambling bytes.
- `a ^ b for a, b in zip(...)` goes through both sequences at the same time and XORs matching bytes together. This single line replaces what would normally be a for-loop.
- `wep_frame = IV + ciphertext` builds the final packet. The IV is placed at the front unencrypted because the receiver needs it to regenerate the keystream.

Decryption and checking side:

- `wep_frame[:3]` takes the first 3 bytes (the IV). `wep_frame[3:]` takes everything after (the encrypted data).
- The receiver generates the same keystream by combining the extracted IV with the shared secret key and running KSA and PRGA again.
- XORing the encrypted data with this keystream gives back the original message plus checksum.
- `decrypted[:-4]` is the recovered message. `decrypted[-4:]` are the last 4 bytes — the received checksum.
- The program recomputes the checksum from the recovered message and compares it with the received one. If they match, the data arrived intact and decryption worked correctly.

7 WEP Weaknesses

Even though WEP was a reasonable idea at the time, it has serious problems:

- **The IV is too short.** With only 24 bits, there are about 16 million possible IV values. On a busy network these run out within hours and the same keystream gets reused, making it easy for attackers to recover the data.
- **The key can be cracked.** Some IV values cause the RC4 algorithm to accidentally reveal parts of the secret key. A tool called Aircrack-ng can recover a WEP key by collecting around 40,000 packets.
- **The checksum can be faked.** CRC-32 is a simple mathematical function, so an attacker can change the encrypted data and adjust the checksum to match without ever knowing the key.
- **No real authentication.** During login, the network sends a challenge and the device sends back an encrypted response. An attacker who watches this exchange can use it to join the network without knowing the key.

Because of these problems, WEP was officially retired in 2004 and replaced by WPA2, which uses a much stronger encryption algorithm (AES) and a better integrity check.

8 Conclusion

In this lab we went through WEP encryption one step at a time using the message "10101010". We computed the checksum, built the RC4 key, generated the keystream, and scrambled the data with XOR to produce the final encrypted packet. We then decrypted it and confirmed the checksum matched, proving the process worked correctly. The Python program automated all these steps and produced identical results, confirming our manual calculations.